SQL Window Functions - Questions

Table: `ecommerce_orders` (300 orders, Jan - Dec 2024)
Columns: order_id, order_date, customer_id, customer_name, city, state, category, product_name, quantity, unit_price, discount_pct, total_amount, payment_mode, order_status
Note: Delivered order = `order_status = 'Delivered'`. Revenue / spend = sum of `total_amount`. Round averages and percentages to 2 decimals.

Section A : OVER()

An empty OVER () makes the window the WHOLE table. Every row keeps its own details and also sees the overall total, average or count.

Q1. For every order, show the total number of orders in the table.

Hint: `COUNT(*) OVER ()`

Note: Unlike GROUP BY, a window function keeps every row. An empty OVER () means 'look at the whole table'.

Answer should display like this:

order_id	customer_name	total_amount	total_orders
1	Isha Reddy	1398	300
2	Sakshi Singh	1499	300
3	Shreya Nair	854	300

... 300 rows in total

Q2. For each delivered order, show the company's average order value, and label the order 'High Value' if it is above the average, otherwise 'Normal'.

Hint: `ROUND(AVG(total_amount) OVER (), 2)` and `CASE WHEN total_amount > AVG(total_amount) OVER () THEN ... ELSE ... END`

Note: You can use a window function inside CASE WHEN, just like a normal column.

Answer should display like this:

order_id	customer_name	total_amount	company_aov	order_type
1	Isha Reddy	1398	2800.29	Normal
2	Sakshi Singh	1499	2800.29	Normal
3	Shreya Nair	854	2800.29	Normal

... 258 rows in total

Q3. For every order, show the total number of orders, the total units sold, and the order's % of the total units.

Hint: `COUNT(*) OVER (), SUM(quantity) OVER (), and quantity * 100.0 / SUM(quantity) OVER ()`

Note: Multiply by 100.0 (not 100) so MySQL does not lose the decimals.

Answer should display like this:

order_id	product_name	quantity	total_orders	total_units	pct_of_units
1	Leather Wallet	2	300	508	0.39
2	Denim Jeans	1	300	508	0.20
3	Cotton Kurta	1	300	508	0.20

... 300 rows in total

Section B : PARTITION BY

PARTITION BY splits the rows into groups (category, city, customer, month). The calculation restarts for each group, but no rows are removed.

Q4. For each delivered order, show the highest and the lowest order amount of the year.

Hint: `MAX(total_amount) OVER () and MIN(total_amount) OVER ()`

Note: The WHERE clause runs first, so the window only sees Delivered orders.

Answer should display like this:

order_id	product_name	total_amount	highest_order	lowest_order
1	Leather Wallet	1398	15996	269
2	Denim Jeans	1499	15996	269
3	Cotton Kurta	854	15996	269

... 258 rows in total

Q5. For each delivered order, show the total revenue of its category and the order's % share of that category.

Hint: `SUM(total_amount) OVER (PARTITION BY category)`

Note: PARTITION BY restarts the calculation for each category. The % values inside one category add up to 100.

Answer should display like this:

order_id	category	product_name	total_amount	category_revenue	pct_of_category
15	Beauty	Perfume	1998	35055	5.70
25	Beauty	Hair Dryer	1424	35055	4.06
26	Beauty	Sunscreen SPF 50	1996	35055	5.69

... 258 rows in total

Q6. For each delivered order, show the average order value of its city.

Hint: `ROUND(AVG(total_amount) OVER (PARTITION BY city), 2)`

Note: Every order of the same city shows the same city_avg.

Answer should display like this:

order_id	city	customer_name	total_amount	city_avg
2	Ahmedabad	Sakshi Singh	1499	2928.42
9	Ahmedabad	Mohit Banerjee	3199	2928.42
12	Ahmedabad	Sakshi Singh	2799	2928.42

... 258 rows in total

Q7. For each delivered order, show the customer's number of orders and the customer's total spend (lifetime value).

Hint: `COUNT(*) OVER (PARTITION BY customer_id) and SUM(total_amount) OVER (PARTITION BY customer_id)`

Note: Partition by customer_id, not customer_name: two customers can share the same name.

Answer should display like this:

customer_id	customer_name	order_id	total_amount	customer_orders	lifetime_value
C001	Harsh Verma	162	664	1	664
C003	Kriti Gupta	47	1999	4	23490
C003	Kriti Gupta	76	9996	4	23490

... 258 rows in total

Q8. For each delivered order, show its month (YYYY-MM), the total revenue of that month, and the order's % of the month.

Hint: `DATE_FORMAT(order_date, '%Y-%m') AS order_month, then PARTITION BY DATE_FORMAT(order_date, '%Y-%m')`

Note: You cannot use the alias order_month inside OVER (). Repeat the full expression.

Answer should display like this:

order_month	order_id	total_amount	month_revenue	pct_of_month
2024-01	1	1398	85206	1.64
2024-01	2	1499	85206	1.76
2024-01	3	854	85206	1.00

... 258 rows in total

Section C : ROW_NUMBER()

ROW_NUMBER() numbers rows 1, 2, 3 ... in the order you choose. Every row gets a different number, even when values tie.

Q9. For each delivered order, show the highest order amount in its category.

Hint: `MAX(total_amount) OVER (PARTITION BY category)`

Note: Orders where total_amount = category_max are the biggest orders of their category.

Answer should display like this:

order_id	category	product_name	total_amount	category_max
15	Beauty	Perfume	1998	4047
25	Beauty	Hair Dryer	1424	4047
26	Beauty	Sunscreen SPF 50	1996	4047

... 258 rows in total

Q10. Number each customer's orders by date (1 = first order).

Hint: `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date, order_id)`

Note: ROW_NUMBER needs ORDER BY inside OVER (). PARTITION BY makes the numbering restart for each customer.

Answer should display like this:

customer_id	customer_name	order_id	order_date	purchase_no
C001	Harsh Verma	30	2024-02-04	1
C001	Harsh Verma	162	2024-07-23	2
C003	Kriti Gupta	47	2024-02-17	1

... 300 rows in total

Q11. Give every order a serial number by order date (oldest = 1). If dates tie, the lower order_id comes first.

Hint: `ROW_NUMBER() OVER (ORDER BY order_date, order_id)`

Note: Without order_id as a tie-breaker, orders on the same date could get different numbers each time you run the query.

Answer should display like this:

order_id	order_date	customer_name	serial_no
1	2024-01-01	Isha Reddy	1
2	2024-01-03	Sakshi Singh	2
3	2024-01-03	Shreya Nair	3

... 300 rows in total

Q12. Number the orders inside each city, newest order first (1 = latest). If dates tie, the higher order_id comes first.

Hint: `ROW_NUMBER() OVER (PARTITION BY city ORDER BY order_date DESC, order_id DESC)`

Note: DESC puts the newest order first, so it gets number 1.

Answer should display like this:

city	order_id	order_date	city_order_no
Ahmedabad	299	2024-12-29	1
Ahmedabad	292	2024-12-18	2
Ahmedabad	277	2024-12-02	3

... 300 rows in total

Q13. Number the delivered orders inside each category from the biggest amount to the smallest. If amounts tie, the lower order_id comes first.

Hint: `ROW_NUMBER() OVER (PARTITION BY category ORDER BY total_amount DESC, order_id)`

Note: Tied amounts still get different numbers with ROW_NUMBER. That is the difference from RANK.

Answer should display like this:

category	order_id	total_amount	position_in_category
Beauty	37	4047	1
Beauty	282	2998	2
Beauty	212	2848	3

... 258 rows in total

Section D : RANK()

RANK() gives tied values the same rank and then SKIPS numbers (1, 1, 3).

Q14. Rank the products inside each category by delivered revenue using RANK().

Hint: `GROUP BY category, product_name, then RANK() OVER (PARTITION BY category ORDER BY SUM(total_amount) DESC)`

Note: A window function can use an aggregate like SUM(total_amount). GROUP BY runs first, then the window.

Answer should display like this:

category	product_name	product_revenue	rank_in_category
Beauty	Hair Dryer	14240	1
Beauty	Perfume	8641	2
Beauty	Sunscreen SPF 50	5788	3

... 30 rows in total

Q15. Rank all delivered orders by amount, highest first, using RANK().

Hint: `RANK() OVER (ORDER BY total_amount DESC)`

Note: Tied amounts share a rank and the next rank is skipped: 1, 2, 2, 4.

Answer should display like this:

order_id	customer_name	total_amount	amount_rank
20	Bhavna Verma	15996	1
23	Shreya Verma	12596	2
90	Kavya Iyer	11998	3

... 258 rows in total

Q16. Rank the delivered orders inside each city by amount, highest first, using RANK().

Hint: `RANK() OVER (PARTITION BY city ORDER BY total_amount DESC)`

Note: The rank starts again at 1 for every city.

Answer should display like this:

city	order_id	total_amount	city_rank
Ahmedabad	20	15996	1
Ahmedabad	205	8996	2
Ahmedabad	40	7598	3

... 258 rows in total

Section E : DENSE_RANK()

DENSE_RANK() gives tied values the same rank and does NOT skip (1, 1, 2).

Q17. Find each customer's total delivered spend and rank the customers, highest first, using RANK().

Hint: `GROUP BY customer_id, customer_name, then RANK() OVER (ORDER BY SUM(total_amount) DESC)`

Note: One row per customer. Customers with the same total share a rank.

Answer should display like this:

customer_id	customer_name	total_spent	spend_rank
C012	Saurabh Pillai	50246	1
C043	Sakshi Singh	49099	2
C050	Bhavna Joshi	41375	3

... 52 rows in total

Q18. Rank the delivered orders inside each category by amount, highest first, using DENSE_RANK().

Hint: `DENSE_RANK() OVER (PARTITION BY category ORDER BY total_amount DESC)`

Note: DENSE_RANK has no gaps: 1, 2, 2, 3. Level 2 is the second highest different amount.

Answer should display like this:

category	order_id	total_amount	amount_level
Beauty	37	4047	1
Beauty	282	2998	2
Beauty	212	2848	3

... 258 rows in total

Q19. Rank the cities inside each state by delivered revenue using DENSE_RANK(), and show the state's revenue.

Hint: `GROUP BY state, city, then SUM(SUM(total_amount)) OVER (PARTITION BY state) and DENSE_RANK() OVER (PARTITION BY state ORDER BY SUM(total_amount) DESC)`

Note: SUM(SUM(...)) means: add up the city totals inside each state.

Answer should display like this:

state	city	city_revenue	state_revenue	city_rank_in_state
Delhi	Delhi	21061	21061	1
Gujarat	Ahmedabad	131779	131779	1
Karnataka	Bengaluru	124989	133184	1

... 12 rows in total

Q20. For delivered orders, show ROW_NUMBER, RANK and DENSE_RANK side by side, ordered by amount (highest first). Use order_id to break ties in ROW_NUMBER.

Hint: `Three window functions in one SELECT, all with ORDER BY total_amount DESC`

Note: Look at tied amounts: ROW_NUMBER 1, 2, 3 | RANK 1, 1, 3 | DENSE_RANK 1, 1, 2.

Answer should display like this:

order_id	total_amount	row_num	rnk	dense_rnk
20	15996	1	1	1
23	12596	2	2	2
90	11998	3	3	3

... 258 rows in total